

Final Project

Michael Drake

2026-04-29

```
library(tidyverse)
library(ggrepel)
library(fabletools)
library(corrplot)
library(GGally)
library(patchwork)
library(caret)
library(pls)
library(impute)
library(mlbench)
library(randomForest)
library(party)
library(rpart)
library(rpart.plot)
library(Cubist)
library(gbm)
library(RWeka)
library(ipred)
library(AppliedPredictiveModeling)
library(readxl)
library(ggcorrplot)
library(mice)
library(missForest)
theme_set(theme_bw())
```

Load Data

```
train_data = read_excel("Project2/StudentData.xlsx")
```

Look for Near Zero Variance Predictors

```
nearZeroVar(train_data)
```

```
## [1] 13
```

```
colnames(train_data[,nearZeroVar(train_data)])
```

```
## [1] "Hyd Pressure1"
```

```
# Gets Number of Each Unique Value  
train_data |>  
  group_by(`Hyd Pressure1`) |>  
  summarize(n = n()) |>  
  arrange(desc(n))
```

```
## # A tibble: 246 x 2  
##   `Hyd Pressure1`     n  
##           <dbl> <int>  
## 1             0    840  
## 2          12.4     27  
## 3          12.8     22  
## 4          12.6     21  
## 5          -0.8     20  
## 6          13.2     20  
## 7          13.8     20  
## 8          14.6     20  
## 9           0.2     19  
## 10          16      19  
## # i 236 more rows
```

```
# Filters Unique Values that Appear More than Once  
train_data |>  
  group_by(`Hyd Pressure1`) |>  
  summarize(n = n()) |>  
  arrange(desc(n)) |>  
  filter(n > 1)
```

```
## # A tibble: 214 x 2  
##   `Hyd Pressure1`     n  
##           <dbl> <int>  
## 1             0    840  
## 2          12.4     27  
## 3          12.8     22  
## 4          12.6     21  
## 5          -0.8     20  
## 6          13.2     20  
## 7          13.8     20  
## 8          14.6     20  
## 9           0.2     19  
## 10          16      19  
## # i 204 more rows
```

Looking at what variables have a near zero variance, only 1 column has a near zero variance: **Hyd Pressure1**. Initially, this suggests that this variable could be excluded from any potential models. However, looking at this predictor more closely, there are 246 unique values, with 214 of these values occurring more than once. This suggests that there is enough variance within this predictor to keep it within the dataset.

Look at Number of Missing Values

```
train_data |> summarise(across(everything(), ~sum(is.na(.))))
```

```
## # A tibble: 1 x 33
##   `Brand Code` `Carb Volume` `Fill Ounces` `PC Volume` `Carb Pressure`
##         <int>         <int>         <int>         <int>         <int>
## 1          120             10             38             39             27
## # i 28 more variables: `Carb Temp` <int>, PSC <int>, `PSC Fill` <int>,
## #   `PSC CO2` <int>, `Mnf Flow` <int>, `Carb Pressure1` <int>,
## #   `Fill Pressure` <int>, `Hyd Pressure1` <int>, `Hyd Pressure2` <int>,
## #   `Hyd Pressure3` <int>, `Hyd Pressure4` <int>, `Filler Level` <int>,
## #   `Filler Speed` <int>, Temperature <int>, `Usage cont` <int>,
## #   `Carb Flow` <int>, Density <int>, MFR <int>, Balling <int>,
## #   `Pressure Vacuum` <int>, PH <int>, `Oxygen Filler` <int>, ...
```

The above code checks for how many missing values exist within each predictor. The tibble above shows that all but 2 predictors (**Pressure Vacuum** and **Air Pressurer**) have at least 1 missing value. The analysis shows the process for dealing with these missing values.

Imputation Methods

For the purposes of imputation, the training dataset is filtered below to only include observations where there is no missing data. From there, each predictor will randomly have data removed, such that when imputation is run on each, it will be compared to the actual value for accuracy purposes.

```
impute_data = train_data |> filter(if_all(everything(), ~!is.na(.)))
```

```
# Defines Function to Calculate NRMSE that Takes 2 Arrays as Inputs:
# Actual Values and Observed Values
nrmse_calc = function(actual_values, obs_values) {
  rmse = sqrt(mean(as.list((actual_values - obs_values)^2)[[1]]))
  data_range = max(actual_values) - min(actual_values)
  nrmse = rmse / data_range
  return(nrmse)
}
```

Calculating NRMSE

```
# Function that Performs Multiple Imputation Methods on Dataset
imputation = function(df) {

  # Randomly Sets 5% of Each Predictor to NA
  missing_data = df |>
  mutate(across(everything(), function(x) {
```

```

na_indices = sample(seq_along(x), 0.05*round(nrow(df)))
x[na_indices] = NA
return(x)
}))

# Generates List of Row Numbers for Each Predictor that's Missing
missing_rows = lapply(missing_data, \(x) which(is.na(x)))

# Performs Mean Imputation
mean_impute = missing_data |> mutate(across(-`Brand Code`, \(x) case_when(
  is.na(x) ~ mean(x, na.rm = TRUE),
  .default = x)))

# Performs Median Imputation
median_impute = missing_data |> mutate(across(-`Brand Code`, \(x) case_when(
  is.na(x) ~ median(x, na.rm = TRUE),
  .default = x)))

# Performs Mode Imputation
mode_impute = missing_data |> mutate(across(-`Brand Code`, \(x) case_when(
  is.na(x) ~ as.numeric(names(table(x))[which.max(table(x))]),
  .default = x))) |>
mutate(`Brand Code` = case_when(
  is.na(`Brand Code`) ~ names(table(`Brand Code`))[which.max(table(`Brand Code`))],
  .default = `Brand Code`))

# Performs KNN Imputation
knn_impute = bind_cols(missing_data$`Brand Code`,
  as.data.frame(impute.knn(as.matrix(
    missing_data |> select(-`Brand Code`)))$data))

# Performs MICE Imputation
mice_impute = complete(mice(
  missing_data |> mutate(`Brand Code` = as.factor(`Brand Code`)),
  printFlag = FALSE), 1)

# Performs Forest Imputation
forest_impute = missForest(as.data.frame(
  missing_data |> mutate(`Brand Code` = as.factor(`Brand Code`))))$ximp

# Creates Empty Arrays for Each Imputation Method for NRMSE Calculations
mean_nrmse = numeric(length(missing_rows)-1)
median_nrmse = numeric(length(missing_rows)-1)
mode_nrmse = numeric(length(missing_rows)-1)
knn_nrmse = numeric(length(missing_rows)-1)
mice_nrmse = numeric(length(missing_rows)-1)
forest_nrmse = numeric(length(missing_rows)-1)

# Check Accuracy of Numeric Variables Using NRMSE
for (i in 2:length(missing_rows)){

  # Extracts Actual and Imputed Values for Each Predictor
  actual_values = df[missing_rows[[i]],i]

```

```

mean_values = mean_impute[missing_rows[[i]],i]
median_values = median_impute[missing_rows[[i]],i]
mode_values = mode_impute[missing_rows[[i]],i]
knn_values = knn_impute[missing_rows[[i]],i]
mice_values = mice_impute[missing_rows[[i]],i]
forest_values = forest_impute[missing_rows[[i]],i]

# Calculates NRMSE for Each Predictor
mean_nrmse[i-1] = nrmse_calc(actual_values, mean_values)
median_nrmse[i-1] = nrmse_calc(actual_values, median_values)
mode_nrmse[i-1] = nrmse_calc(actual_values, mode_values)
knn_nrmse[i-1] = nrmse_calc(actual_values, knn_values)
mice_nrmse[i-1] = nrmse_calc(actual_values, mice_values)
forest_nrmse[i-1] = nrmse_calc(actual_values, forest_values)
}

# Averages NRMSE Across All Predictors for Each Imputation Method
avg_mean_nrmse = mean(mean_nrmse)
avg_median_nrmse = mean(median_nrmse)
avg_mode_nrmse = mean(mode_nrmse)
avg_knn_nrmse = mean(knn_nrmse)
avg_mice_nrmse = mean(mice_nrmse)
avg_forest_nrmse = mean(forest_nrmse)

# Extracts Actual and Imputed Brand Codes
actual_brand_codes = df[missing_rows[[1]],1]
mode_brand_codes = mode_impute[missing_rows[[1]],1]$`Brand Code`
mice_brand_codes = as.character(mice_impute[missing_rows[[1]],1])
forest_brand_codes = as.character(forest_impute[missing_rows[[1]],1])

# Calculates Accuracy of Imputation Methods for Brand Codes
mode_brand_accuracy = sum(actual_brand_codes == mode_brand_codes) /
  length(mode_brand_codes)
mice_brand_accuracy = sum(actual_brand_codes == mice_brand_codes) /
  length(mice_brand_codes)
forest_brand_accuracy = sum(actual_brand_codes == forest_brand_codes) /
  length(forest_brand_codes)

# Returns All Accuracy Values

return(list("data" = missing_data, "mean" = avg_mean_nrmse,
  "median" = avg_median_nrmse, "mode" = avg_mode_nrmse,
  "knn" = avg_knn_nrmse, "mice" = avg_mice_nrmse,
  "forest" = avg_forest_nrmse, "mode_brand" = mode_brand_accuracy,
  "mice_brand" = mice_brand_accuracy,
  "forest_brand" = forest_brand_accuracy))
}

```

Defining Function

```

# Sets Number of Iterations to Be Run
iter = 10

# Creates Empty Arrays the Length of the Number of Iterations
avg_mean_nrmse = numeric(iter)
avg_median_nrmse = numeric(iter)
avg_mode_nrmse = numeric(iter)
avg_knn_nrmse = numeric(iter)
avg_mice_nrmse = numeric(iter)
avg_forest_nrmse = numeric(iter)
avg_mode_accuracy = numeric(iter)
avg_mice_accuracy = numeric(iter)
avg_forest_accuracy = numeric(iter)
missing_data_list = vector("list", iter)

# Loop that Runs Imputation Function Previously Defined
for (i in 1:iter) {
  missing_data_info = imputation(impute_data)

  missing_data_list[[i]] = missing_data_info$data
  avg_mean_nrmse[i] = missing_data_info$mean
  avg_median_nrmse[i] = missing_data_info$median
  avg_mode_nrmse[i] = missing_data_info$mode
  avg_knn_nrmse[i] = missing_data_info$knn
  avg_mice_nrmse[i] = missing_data_info$mice
  avg_forest_nrmse[i] = missing_data_info$forest
  avg_mode_accuracy[i] = missing_data_info$mode_brand
  avg_mice_accuracy[i] = missing_data_info$mice_brand
  avg_forest_accuracy[i] = missing_data_info$forest_brand
}

# Calculates Average NRMSE and Accuracy based on Outputs from Each Run
# in the Loop Above
mean_nrmse = mean(avg_mean_nrmse)
median_nrmse = mean(avg_median_nrmse)
mode_nrmse = mean(avg_mode_nrmse)
knn_nrmse = mean(avg_knn_nrmse)
mice_nrmse = mean(avg_mice_nrmse)
forest_nrmse = mean(avg_forest_nrmse)
mode_accuracy = mean(avg_mode_accuracy)
mice_accuracy = mean(avg_mice_accuracy)
forest_accuracy = mean(avg_forest_accuracy)

```

Running Loop

```

# Puts NRMSE into Dataframe
nrmse_values = c(mean_nrmse, median_nrmse, mode_nrmse,
                 knn_nrmse, mice_nrmse, forest_nrmse)
nrmse_names = c("mean", "median", "mode", "knn", "mice", "forest")
nrmse_comparison = data.frame("method" = nrmse_names, "nrmse" = nrmse_values)

```

```

# Puts Accuracy Values into Dataframe
accuracy_values = c(mode_accuracy, mice_accuracy, forest_accuracy)
accuracy_names = c("mode", "mice", "forest")
accuracy_comparison = data.frame("method" = accuracy_names,
                                "accuracy" = accuracy_values)

# Sorts NRMSE and Accuracy Values
nrmse_comparison |> arrange(nrmse)

```

Comparing Results

```

##   method    nrmse
## 1 forest 0.1011329
## 2  mice 0.1582512
## 3   knn 0.1884385
## 4   mean 0.2475154
## 5 median 0.2690078
## 6   mode 0.3122949

```

```

accuracy_comparison |> arrange(desc(accuracy))

```

```

##   method accuracy
## 1 forest 0.9742574
## 2  mice 0.8782178
## 3   mode 0.5841584

```

Based on the above simulations, the forest method performs the best when imputing for both the numerical and non-numerical data when tested on data where missing values were randomly placed in each of the predictors. As a result, the forest method is used for imputation on the full training dataset.

```

# Generates Updated Training Data with Forest Imputation Method
updated_training = missForest(as.data.frame(
  train_data |> mutate(`Brand Code` = as.factor(`Brand Code`))))$ximp

# Verifies there are No Missing Values
updated_training |> summarise(across(everything(), ~sum(is.na(.))))

```

```

##   Brand Code Carb Volume Fill Ounces PC Volume Carb Pressure Carb Temp PSC
## 1         0         0         0         0         0         0         0         0
##   PSC Fill PSC CO2 Mnf Flow Carb Pressure1 Fill Pressure Hyd Pressure1
## 1         0         0         0         0         0         0         0
##   Hyd Pressure2 Hyd Pressure3 Hyd Pressure4 Filler Level Filler Speed
## 1         0         0         0         0         0         0
##   Temperature Usage cont Carb Flow Density MFR Balling Pressure Vacuum PH
## 1         0         0         0         0 0         0         0         0
##   Oxygen Filler Bowl Setpoint Pressure Setpoint Air Pressurer Alch Rel Carb Rel
## 1         0         0         0         0         0         0         0
##   Balling Lvl
## 1         0

```

Look at Potential Data Transformations

The below code checks if any data transformations are recommended for the dataset. The Box-Cox method is used.

```
updated_training |>
  mutate(across(where(is.numeric), ~ BoxCoxTrans(.)$lambda)) |>
  head(1)

##   Brand Code Carb Volume Fill Ounces PC Volume Carb Pressure Carb Temp PSC
## 1         B      -2         2      0.7      -0.2      -1.6 0.5
##   PSC Fill PSC CO2 Mnf Flow Carb Pressure1 Fill Pressure Hyd Pressure1
## 1      NA      NA      NA          0.5      -0.7      NA
##   Hyd Pressure2 Hyd Pressure3 Hyd Pressure4 Filler Level Filler Speed
## 1      NA      NA      -0.2         2         2
##   Temperature Usage cont Carb Flow Density MFR Balling Pressure Vacuum PH
## 1      -2         2      1.3      0  2      NA      NA  2
##   Oxygen Filler Bowl Setpoint Pressure Setpoint Air Pressurer Alch Rel Carb Rel
## 1      0.2         2      -2      -2      -2      -2
##   Balling Lvl
## 1      NA
```

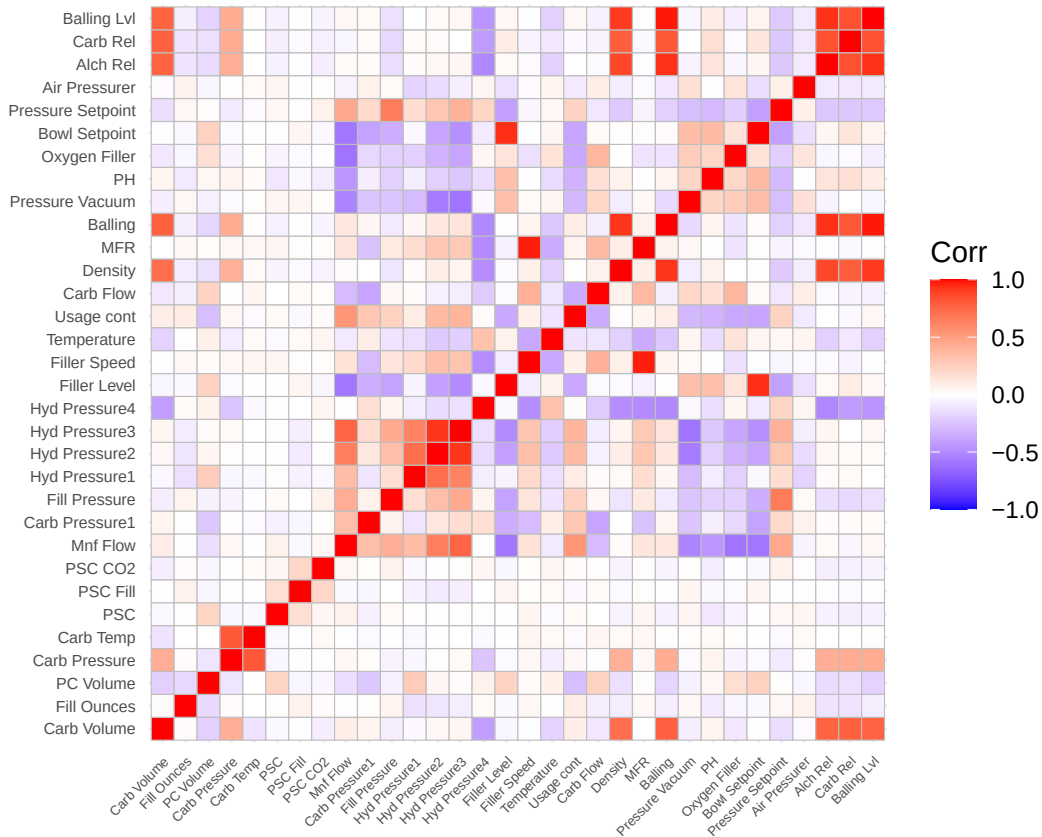
Based on the output above, there are many predictors that would benefit from a transformation. Additionally, the data should be centered and scaled. The code below used `preProcess` to apply all 3 of these steps in one.

```
trans = preProcess(updated_training, method = c("BoxCox", "center", "scale"))
transformed_data = predict(trans, updated_training)
```

Look at Correlation Between Variables

```
# Calculates Correlation Between All Numeric Variables
predictor_cor = cor(transformed_data |> select(-`Brand Code`), use = "complete.obs")

# Plots Correlation Values
ggcorrplot(predictor_cor, method = "square", hc.order = FALSE) +
  theme(axis.text.x = element_text(size = 5),
        axis.text.y = element_text(size = 6))
```

```
# Lists Column Names that Exceed Correlation Values of 0.8 and Recommend
# Being Removed
transformed_data[,findCorrelation(predictor_cor, cutoff = 0.8)] |> colnames()
```

```
## [1] "Hyd Pressure2" "MFR" "Carb Rel" "Air Pressurer"
## [5] "Hyd Pressure4" "Filler Level" "Carb Pressure"
```

```
# Removes Highly Correlated Values
final_train = transformed_data |>
  select(-findCorrelation(predictor_cor, cutoff = 0.8))
```

The above code checks for highly correlated predictors in an attempt to remove redundant predictors from the dataset. The correlation matrix shows there are many predictors that are highly correlated, which is verified with the `findCorrelation` function above. These highly correlated predictors were removed from the dataset. For this exercise, the cutoff was set to an R^2 value of 0.8. The final training dataset after all pre-processing is the `final_train` dataframe above.